

Semaine 1 — Fondamentaux du web & environnement

Polycopié étudiant — Développement web, classe passerelle

Cours 4h

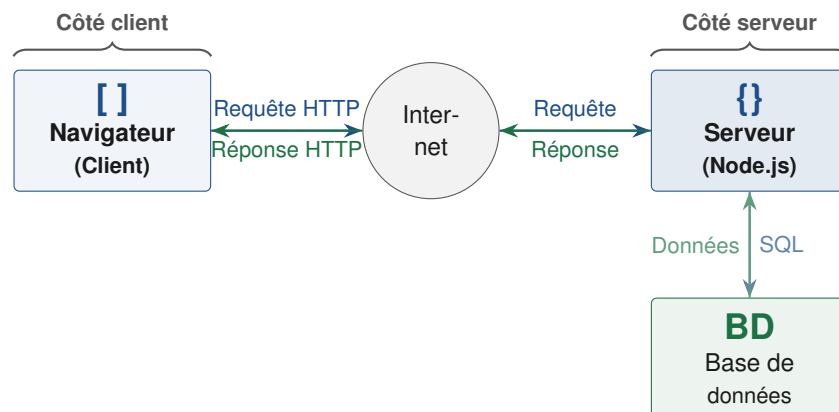
TP 4h

- Comprendre le modèle client/serveur et le fonctionnement du web
- Distinguer les rôles de HTML, CSS et JavaScript
- Distinguer un site statique d'un site dynamique
- Installer et configurer son environnement de développement
- Créer son premier dépôt Git et effectuer un premier commit

1 Comment fonctionne le web ?

Quand tu tapes `https://www.hepia.ch` dans ton navigateur, une série d'échanges invisibles se produit en quelques millisecondes. Comprendre ce mécanisme est le point de départ de tout développeur web.

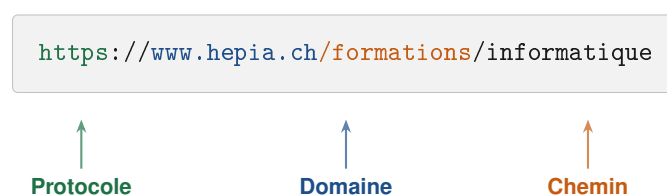
Le modèle client / serveur



Client : programme qui *demande* une ressource (le navigateur).

Serveur : programme qui *répond* à la demande en envoyant des fichiers ou des données. La communication se fait via le protocole **HTTP** (ou **HTTPS** pour la version sécurisée).

Anatomie d'une URL



DNS — de l'adresse au nom

Un ordinateur ne comprend pas `www.hepia.ch` : il a besoin d'une adresse numérique, appelée **adresse IP** (par exemple `192.0.2.34`). Le **DNS** (*Domain Name System*) est le service qui traduit un nom de domaine lisible en adresse IP.

Le DNS fonctionne comme un annuaire : tu connais le *nom* d'une personne (le domaine), l'annuaire te donne son *numéro* (l'adresse IP), et c'est ce numéro qui permet réellement d'établir la communication.

Étapes simplifiées :

1. Le navigateur demande au DNS : « quelle est l'adresse IP de `www.hepia.ch` ? »
2. Le serveur DNS répond avec l'adresse IP correspondante
3. Le navigateur peut alors envoyer sa requête HTTP directement à cette adresse

Les codes de statut HTTP

Code	Signification	Quand ?
200	OK	Tout s'est bien passé
301	Redirection	La page a changé d'adresse
404	Not Found	La page n'existe pas
500	Server Error	Erreur côté serveur

Sites statiques et sites dynamiques

Un **site statique** renvoie toujours le même contenu au client : le fichier HTML existe déjà sur le serveur et est envoyé tel quel.

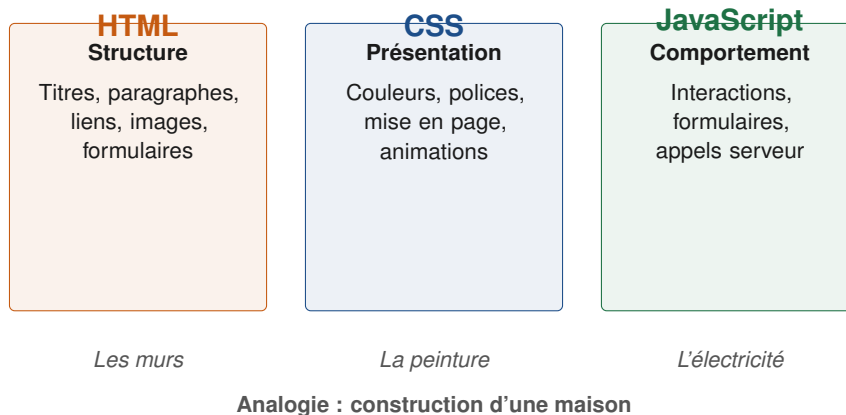
Un **site dynamique** génère le contenu à la demande, souvent à partir d'une base de données : deux visiteurs, ou deux moments différents, peuvent recevoir des pages différentes.

	Site statique	Site dynamique
Contenu	Identique pour tout le monde	Généré à chaque requête
Exemple	Page vitrine simple, portfolio	Forum, boutique en ligne, réseau social
Techniquement	Fichiers HTML/CSS/JS seuls	Serveur applicatif (Node.js) + base de données

Où se situe MiniForum ?

Le projet fil rouge **MiniForum** commence comme un **site statique** (données codées en dur en semaine 1–5), puis devient **dynamique** à partir de la phase 3, lorsqu'il se connecte à une vraie base de données via Node.js/Express.

2 Les trois langages du web



Important — Qui exécute quoi ?

HTML, CSS et JavaScript s'exécutent dans le **navigateur** (côté client).

Node.js s'exécute sur le **serveur** (côté serveur) — découvert en phase 3.

La base de données est aussi côté serveur, jamais accessible directement depuis le navigateur.

Un premier exemple

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>MiniForum</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Bienvenue sur MiniForum</h1>
10  <button id="btn">Cliquez-moi</button>
11  <script src="app.js"></script>
12 </body>
13 </html>
```

Extrait 1: index.html — Structure

```
1 button {
2   background-color: #1D3557; /* bleu marine */
3   color: white;
4   padding: 10px 20px;
5   border: none;
6   border-radius: 4px;
7   cursor: pointer;
8 }
```

Extrait 2: style.css — Présentation

```
1 const btn = document.getElementById('btn');
2 btn.addEventListener('click', () => {
3   alert('Bonjour depuis JavaScript !');
4 });
```

Extrait 3: app.js — Comportement

3 La stack du cours



4 Environnement de développement

Outils à installer

- | | |
|-----------------------------|---|
| • Visual Studio Code | code.visualstudio.com |
| • Node.js LTS (v20+) | nodejs.org |
| • Git | git-scm.com |
| • Compte GitHub | github.com |

Extensions VS Code recommandées

Extension	Utilité
Prettier	Formatage automatique du code
ESLint	Détection d'erreurs JavaScript
Live Server	Rechargement automatique du navigateur
REST Client	Tester les routes API depuis VS Code
GitLens	Visualiser l'historique Git dans l'éditeur

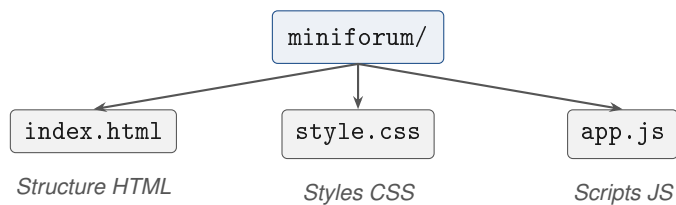
Premiers pas dans le terminal

Git et Node.js s'utilisent en ligne de commande. Quelques commandes suffisent pour naviguer dans les dossiers de ton projet.

Commande	Effet
<code>pwd</code>	Affiche le dossier courant
<code>ls</code> (ou <code>dir</code>)	Liste le contenu du dossier courant
<code>cd nom-dossier</code>	Se déplace dans un sous-dossier
<code>cd ..</code>	Remonte d'un niveau
<code>mkdir nom</code>	Crée un nouveau dossier
<code>touch nom.txt</code>	Crée un fichier vide (macOS/Linux)

Ces commandes seront revues et approfondies dans le cours Linux/Bash donné en parallèle. Il suffit ici de savoir se déplacer et créer un dossier pour démarrer le projet MiniForum.

Structure du projet MiniForum



5 Git — Premiers pas

Git est un outil de **gestion de version** : il enregistre l'historique de chaque modification de ton code. C'est un outil professionnel utilisé dans tous les projets informatiques.

Les quatre zones de Git



Commandes essentielles

```

1 # Initialiser un depot Git
2 git init
3
4 # Voir l'etat des fichiers
5 git status
6
7 # Ajouter tous les fichiers au stage
8 git add .
9
10 # Enregistrer un commit
11 git commit -m "Premier commit : structure de base du MiniForum"
12
13 # Lier au depot GitHub
14 git remote add origin https://github.com/votre-login/miniforum.git
  
```

```
15  
16 # Envoyer sur GitHub  
17 git push -u origin main
```

Extrait 4: Premiers pas avec Git

Un bon message de commit décrit **ce qui a changé**, pas comment.

OK Ajout formulaire création de sujet

NON modif **NON** ca marche **NON** update

6 Lexique de la semaine

Terme	Définition
URL	Adresse permettant de localiser une ressource sur le web
DNS	Service qui traduit un nom de domaine en adresse IP
HTTP / HTTPS	Protocole de communication entre client et serveur (HTTPS = version chiffrée)
Client	Programme qui demande une ressource (le navigateur)
Serveur	Programme qui répond à une demande en fournissant fichiers ou données
Statique	Contenu identique quel que soit le visiteur ou le moment
Dynamique	Contenu généré à la demande, souvent depuis une base de données
Commit	Enregistrement d'un instantané des modifications dans Git
Dépôt (repository)	Dossier suivi par Git, contenant l'historique du projet

Pour aller plus loin

- Un **quiz d'auto-évaluation** sur cette semaine est disponible sur Moodle.
- La **fiche TP** (mise en place de l'environnement et premier dépôt Git) est distribuée séparément : *S01_TP.pdf*.

Résumé de la semaine

- Le web fonctionne sur un modèle **client / serveur** via le protocole **HTTP**
- Le **DNS** traduit un nom de domaine en adresse IP
- **HTML** structure, **CSS** présente, **JavaScript** rend interactif
- Un site **statique** renvoie toujours le même contenu, un site **dynamique** le génère à la demande
- **Git** : add → commit → push pour versionner son code
- **MiniForum** sera le projet fil rouge développé progressivement sur 15 semaines

Pour la semaine 2

- Vérifier que VS Code, Node.js et Git sont correctement installés
- Créer son compte GitHub si ce n'est pas déjà fait
- Avoir son dépôt `miniforum/` initialisé avec les trois fichiers de base et poussé sur GitHub